

# 用户调度问题

## 题目描述

在 通信系统 中，一个常见的问题是对用户进行不同策略的调度，会得到不同的系统消耗和性能。

假设当前有n个待串行调度用户，每个用户可以使用A/B/C三种 不同的 调度策略，不同的策略会消耗不同的系统资源。请你根据如下规则进行用户调度，并返回总的消耗资源数。

规则：

- 1. 相邻的用户不能使用相同的调度策略，例如，第1个用户使用了A策略，则第2个用户只能使用B或者C策略。
- 2. 对单个用户而言，不同的调度策略对系统资源的消耗可以归一化后抽象为数值。例如，某用户分别使用A/B/C策略的系统消耗分别为15/8/17。
- 3. 每个用户依次选择当前所能选择的对系统资源消耗最少的策略（局部最优），如果有多个满足要求的策略，选最后一个。

## 输入描述

第一行表示用户个数n

接下来每一行表示一个用户分别使用三个策略的系统消耗resA resB resC

## 输出描述

最优策略组合下的总的系统资源消耗数

## 用例

|    |              |
|----|--------------|
| 输入 | 3<br>15 8 17 |
|----|--------------|

|    |                                                          |
|----|----------------------------------------------------------|
|    | 12 20 9<br>11 7 5                                        |
| 输出 | 24                                                       |
| 说明 | 1号用户使用B策略，2号用户使用C策略，3号用户使用B策略。系统资源消耗: $8 + 9 + 7 = 24$ 。 |

## 局部最优解法

本题第3个条件如下：

每个用户依次选择当前所能选择的对系统资源消耗最少的策略（**局部最优**），如果有多个满足要求的策略，选最后一个。

其中，每个用户选择得策略，只需要满足局部最优即可，比如题目用例：

第1个用户，局部最优应该选[15, 8, 17]中最小值8

第2个用户，由于受限于相邻用户不能选相同位置得策略，因此只能选择[12, 9]中局部最优的9

第3个用户，由于受限于相邻用户不能选相同位置得策略，因此只能选择[11, 7]中局部最优的7

因此，最终总消耗为： $8 + 9 + 7 = 24$

因此，有了这个局部最优的条件，本题的难度大大降低了。

## JS算法源码

```
1  /* JavaScript Node ACM模式 控制台输入获取 */
2  const readline = require("readline");
3
4  const rl = readline.createInterface({
5    input: process.stdin,
6    output: process.stdout,
```

运行

```
7 | }); 8 |
9 | const lines = [];
10 | let n;
11 | rl.on("line", (line) => {
12 |   lines.push(line);
13 |
14 |   if (lines.length === 1) {
15 |     n = parseInt(lines[0]);
16 |   }
17 |
18 |   if (n !== undefined && lines.length === n + 1) {
19 |     lines.shift();
20 |     const res = lines.map((line) => line.split(" ").map(Number));
21 |
22 |     console.log(getResult(n, res));
23 |
24 |     lines.length = 0;
25 |   }
26 | });
27 |
28 | function getResult(n, res) {
29 |   let last = -1;
30 |   let sum = 0;
31 |
32 |   for (let i = 0; i < n; i++) {
33 |     last = getMinEleIdx(res[i], last);
34 |     sum += res[i][last];
35 |   }
36 |
37 |   return sum;
38 | }
39 |
40 | function getMinEleIdx(arr, excludeIdx) {
41 |   let minEleVal = Infinity;
42 |   let minEleIdx = -1;
43 |
```

```

44   for (let i = 0; i < arr.length; i++) {45       if (i == excludeIdx) continue;
46
47       if (arr[i] <= minEleVal) {
48           minEleVal = arr[i];
49           minEleIdx = i;
50       }
51   }
52
53   return minEleIdx;
54 }

```

## Java算法源码

```

1   import java.util.Scanner;
2
3   public class Main {
4       public static void main(String[] args) {
5           Scanner sc = new Scanner(System.in);
6
7           int n = sc.nextInt();
8
9           int[][] res = new int[n][3];
10          for (int i = 0; i < n; i++) {
11              res[i][0] = sc.nextInt();
12              res[i][1] = sc.nextInt();
13              res[i][2] = sc.nextInt();
14          }
15
16          System.out.println(getResult(n, res));
17      }
18
19      public static int getResult(int n, int[][] res) {
20          int last = -1;
21          int sum = 0;
22

```

```

23     for (int i = 0; i < n; i++) {
24         last = getMinEleIdx(res[i], last);
25         sum += res[i][last];
26     }
27
28     return sum;
29 }
30
31 public static int getMinEleIdx(int[] arr, int excludeIdx) {
32     int minEleVal = Integer.MAX_VALUE;
33     int minEleIdx = -1;
34
35     for (int i = 0; i < arr.length; i++) {
36         if (i == excludeIdx) continue;
37
38         if (arr[i] <= minEleVal) {
39             minEleVal = arr[i];
40             minEleIdx = i;
41         }
42     }
43
44     return minEleIdx;
45 }
46 }

```

## Python算法源码

```

1 import sys
2
3 # 输入获取
4 n = int(input())
5 res = [list(map(int, input().split())) for _ in range(n)]
6
7
8 def getMinEleIdx(arr, excludeIdx):
9     minEleVal = sys.maxsize

```

```

10 |     minEleIdx = -1
11 |
12 |     for i in range(len(arr)):
13 |         if i == excludeIdx:
14 |             continue
15 |
16 |         if arr[i] <= minEleVal:
17 |             minEleVal = arr[i]
18 |             minEleIdx = i
19 |
20 |     return minEleIdx
21 |
22 |
23 | # 算法入口
24 | def getResult():
25 |     last = -1
26 |     total = 0
27 |
28 |     for i in range(n):
29 |         last = getMinEleIdx(res[i], last)
30 |         total += res[i][last]
31 |
32 |     return total
33 |
34 |
35 | # 算法调用
36 | print(getResult())

```

## C算法源码

```

1 | #include <stdio.h>
2 | #include <limits.h>
3 |
4 | int getMinEleIdx(int arr[], int arr_size, int excludeIdx);
5 | int getResult(int n, int res[][3]);

```

```
6 | 7 | int main() {
7 |   int n;
8 |   scanf("%d", &n);
9 |
10 |
11 |   int res[n][3];
12 |   for(int i=0; i<n; i++) {
13 |       scanf("%d %d %d", &res[i][0], &res[i][1], &res[i][2]);
14 |   }
15 |
16 |   printf("%d\n", getResult(n, res));
17 |
18 |   return 0;
19 | }
20 |
21 | int getResult(int n, int res[n][3]) {
22 |     int last = -1;
23 |     int sum = 0;
24 |
25 |     for(int i=0; i<n; i++) {
26 |         last = getMinEleIdx(res[i], 3, last);
27 |         sum += res[i][last];
28 |     }
29 |
30 |     return sum;
31 | }
32 |
33 | int getMinEleIdx(int arr[], int arr_size, int excludeIdx) {
34 |     int minEleVal = INT_MAX;
35 |     int minEleIdx = -1;
36 |
37 |     for(int i=0; i<arr_size; i++) {
38 |         if(i == excludeIdx) continue;
39 |
40 |         if(arr[i] <= minEleVal) {
41 |             minEleVal = arr[i];
42 |             minEleIdx = i;
43 |         }
```

```
44 |     }  
    |     45 |  
46 |     return minEleIdx;  
47 | }
```

## C++ 算法源码

```
1  #include <bits/stdc++.h>  
2  using namespace std;  
3  
4  int getMinEleIdx(const int arr[], int arr_size, int excludeIdx) {  
5      int minEleVal = INT_MAX;  
6      int minEleIdx = -1;  
7  
8      for (int i = 0; i < arr_size; i++) {  
9          if (i == excludeIdx) continue;  
10  
11         if (arr[i] <= minEleVal) {  
12             minEleVal = arr[i];  
13             minEleIdx = i;  
14         }  
15     }  
16  
17     return minEleIdx;  
18 }  
19  
20 int solution(int n, int res[][3]) {  
21     int last = -1;  
22     int sum = 0;  
23  
24     for (int i = 0; i < n; i++) {  
25         last = getMinEleIdx(res[i], 3, last);  
26         sum += res[i][last];  
27     }  
28 }
```



```

29 |     return sum;
    |         30 | }
31 |
32 | int main() {
33 |     int n;
34 |     cin >> n;
35 |
36 |     int res[n][3];
37 |     for (int i = 0; i < n; i++) {
38 |         cin >> res[i][0] >> res[i][1] >> res[i][2];
39 |     }
40 |
41 |     cout << solution(n, res) << endl;
42 |
43 |     return 0;
44 | }

```

## 全局最优解法（不符合本题要求，仅供学习参考）

如果本题没有下面这个要求

每个用户依次选择当前所能选择的对系统资源消耗最少的策略（局部最优），如果有多个满足要求的策略，选最后一个。

则要求的最少系统资源消耗数就是要全局最优的。

此时，我们可以通过回溯算法，求出所有组合情况，比较得出其中最小资源消耗的组合。

而本题回溯算法求解组合逻辑如下：

按照题目意思，用户是串行调度的，因此执行顺序不能改变，比如题目用例中，第一层必须选15 8 17，第二层必须选12 20 9，第三层必须选11 7 5。

另外，题目要求相邻层级间不能使用相同系统消耗策略，即：

- 第一层选了15，

- 第二层就不能选12，但是可以选20或9，如果第二层选了20，
- 则第三层就不能选7，但是可以选11或5.

因此回溯算法的递归层数、每层节点的筛选条件我们就都有了。

## JavaScript 算法源码

```
1  /* JavaScript Node ACM模式 控制台输入获取 */
2  const readline = require("readline");
3
4  const rl = readline.createInterface({
5    input: process.stdin,
6    output: process.stdout,
7  });
8
9  const lines = [];
10 let m;
11 rl.on("line", (line) => {
12   lines.push(line);
13
14   if (lines.length === 1) {
15     m = parseInt(lines[0]);
16   }
17
18   if (m !== undefined && lines.length === m + 1) {
19     lines.shift();
20     const res = lines.map((line) => line.split(" ").map(Number));
21     const ans = [Infinity];
22     dfs(res, m, 0, -1, 0, ans);
23     console.log(ans[0]);
24
25     lines.length = 0;
26   }
27 });
28
```

运行

```

29 | function dfs(res, m, level, index, total, ans) {
30 |     if (level == m) {
31 |         ans[0] = Math.min(ans[0], total);
32 |         return;
33 |     }
34 |
35 |     const r = res[level];
36 |     for (let i = 0; i < r.length; i++) {
37 |         if (i !== index) {
38 |             dfs(res, m, level + 1, i, total + r[i], ans);
39 |         }
40 |     }
41 | }

```

## Java算法源码

```

1 | import java.util.Scanner;
2 |
3 | public class Main {
4 |     public static void main(String[] args) {
5 |         Scanner sc = new Scanner(System.in);
6 |
7 |         int m = sc.nextInt();
8 |
9 |         int[][] res = new int[m][3];
10 |        for (int i = 0; i < m; i++) {
11 |            res[i][0] = sc.nextInt();
12 |            res[i][1] = sc.nextInt();
13 |            res[i][2] = sc.nextInt();
14 |        }
15 |
16 |        int[] ans = {Integer.MAX_VALUE};
17 |        dfs(res, m, 0, -1, 0, ans);
18 |        System.out.println(ans[0]);

```

```

19 | }
20 |
21 | public static void dfs(int[][] res, int m, int level, int index, int total, int[] ans) {
22 |     if (level == m) {
23 |         ans[0] = Math.min(ans[0], total);
24 |         return;
25 |     }
26 |
27 |     int[] r = res[level];
28 |     for (int i = 0; i < r.length; i++) {
29 |         if (i != index) {
30 |             dfs(res, m, level + 1, i, total + r[i], ans);
31 |         }
32 |     }
33 | }
34 | }

```

## Python算法源码

```

1 | import sys
2 |
3 | # 输入获取
4 | m = int(input())
5 | res = [list(map(int, input().split())) for _ in range(m)]
6 |
7 |
8 | # 算法入口
9 | def dfs(level, index, total, ans):
10 |     if level == m:
11 |         ans[0] = min(ans[0], total)
12 |         return
13 |
14 |     r = res[level]
15 |     for i in range(len(r)):
16 |         if i != index:

```

```
17         dfs(level + 1, i, total + r[i], ans)18
19
20 # 算法调用
21 ans = [sys.maxsize]
22 dfs(0, -1, 0, ans)
23 print(ans[0])
```